

课程笔记

[LLM Agents F24] Enterprise Trends for Generative AI and Building Successful Agents —Burak Gokturk

基于公开课程资料整理

2024 年 9 月 30 日



BURAK GOKTURK
VP, ML, SYSTEMS, AND
CLOUD AI RESEARCH



视频作者/频道: Berkeley RDI

发布日期: 2024 年 9 月 30 日

视频时长: 约 70 分钟

视频链接: <https://www.youtube.com/live/Sy1psHS3w3I>

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 引言：企业视角下的 GenAI 趋势	2
1.1 本章小结	2
2 企业 GenAI 的关键趋势	2
2.1 AI 民主化	2
2.2 数据需求降低	2
2.3 模型快速迭代	2
2.4 多模态融合	3
2.5 成本与延迟持续下降	3
2.6 本章小结	3
3 GenAI 技术演进时间线	3
3.1 2018-2021: foundation model 时代	3
3.2 2022-2023: Agent 与 multi-turn	3
3.3 2024 之后: 实时、可控、合规	4
3.4 本章小结	4
4 企业 GenAI 典型实践	4
4.1 金融服务中的智能助理	4
4.2 制造与供应链的 GenAI Agent	4
4.3 本章小结	5
5 搜索 + LLM: 企业的杀手级应用	5
5.1 本章小结	5
6 构建 AI Agent 的关键模块	5
6.1 Vertex AI 平台架构	5
6.2 Gemini 的差异化	5
6.3 Planner 与 Memory 模块	6
6.4 Tool Orchestration 与 Workflow	6
6.5 本章小结	6
7 AI Agent 的可观测性与安全控制	7
7.1 监控指标与报警策略	7
7.2 安全治理与 guardrail 设计	7
7.3 本章小结	7
8 AI Agent 的部署与运营	8
8.1 成本与延迟 trade-off	8
8.2 数据与用户反馈闭环	8
8.3 本章小结	8
9 对学生和创业者的建议	9
9.1 本章小结	9

10 LLM Agent 的评估与实验室	9
10.1 定量评估指标	9
10.2 Human-in-the-loop 实验室	9
10.3 本章小结	9
11 治理与法规对接	10
11.1 政策与合规需求	10
11.2 技术与治理结合	10
11.3 本章小结	10
12 组织与投资的启示	10
12.1 人才结构与文化	10
12.2 资本分配与项目组合	10
12.3 本章小结	11
13 关键幻灯片回顾	11
13.1 封面与主题定位	11
13.2 趋势图谱与 timeline	11
13.3 实时性与可观察性展示	12
13.4 本章小结	12
14 Agent Workflow 深度剖析	12
14.1 发现与理解	12
14.2 规划与工具选择	12
14.3 执行与复核	12
14.4 收尾与迭代	12
14.5 本章小结	12
15 治理案例：监管 playbook	13
15.1 金融监管 playbook	13
15.2 医疗与政府的额外 guardrail	13
15.3 本章小结	13
16 附录：Prompt 与 Tool 模板	13
16.1 Prompt Template 架构	13
16.2 Tool Manifest 示例	13
16.3 Failure Tag 标签体系	14
16.4 本章小结	14
17 Q&A 与辩题	14
17.1 用户问答速览	14
17.2 辩题与反思	14
17.3 本章小结	14

18 运营检查表	15
18.1 Deployment Readiness Checklist	15
18.2 Post-launch Monitoring	15
18.3 本章小结	15
19 未来展望与行动建议	15
19.1 未来 12 个月的路线	15
19.2 战略性建议	15
19.3 本章小结	15
20 术语速查	16
20.1 核心术语表	16
20.2 应用与维护建议	16
21 附加素材与灵感	16
21.1 组合的思路	16
21.2 引用与灵感	16
21.3 本章小结	16
22 总结与延伸	17
22.1 课程全局总结表	17
22.2 进一步行动与反思	17
22.3 拓展阅读	17

1 引言：企业视角下的 GenAI 趋势

Burak Gokturk 是 Google 的工程副总裁。本讲不深入算法细节，而是从**企业市场**的宏观视角审视 GenAI 的发展趋势，为学生和创业者提供实用的方向指引。

从 PhD 到产业的视角转变

Burak Gokturk 以自己的 PhD 经历开场：当年神经网络被认为“不如 SVM”，训练 50 个病人的 CT 数据需要 42 秒就觉得太慢。如今大模型训练动辄数周。一个有趣的担忧是：训练周期过长导致研究者获取直觉的速度变慢了。

1.1 本章小结

AI 的发展速度远超预期，从学术界到企业界都在经历范式转变。

2 企业 GenAI 的关键趋势

2.1 AI 民主化

开发者取代 AI 专家

过去构建 AI 应用需要数据科学家和 ML 专家；如今任何开发者——甚至中学生——都可以用 LLM API 构建 AI 应用。这将 AI 开发者群体扩大了 100 倍以上，是 AI 应用爆发式增长的根本原因。

AI 民主化的背后是两层力量：其一是越来越多的基础模型开放 API，其二是低代码/无代码工具把 prompt 设计、数据清洗与 Ops 流程封装成可视化工作流。这两者共同把 AI 应用从少数 data scientist 转移到大批软件工程师与业务团队。

2.2 数据需求降低

- 过去企业需要花费数月甚至数年收集标注数据
- 大模型已经包含海量预训练知识，fine-tuning 所需数据量大幅减少
- 企业可以在数天内启动 AI 项目，而非数年

减少数据需求不代表可以跳过数据治理。企业仍需建立「数据目录」「数据质量评分」和「差异化存取」策略，防止模型训练时吸入违规内容。很多企业选择先用公开数据做验证，再用小批高质量数据做微调。

2.3 模型快速迭代

- 每隔几周就有新模型发布，排行榜不断变化
- 企业关注**平台**而非单个模型——需要能灵活切换模型的基础设施
- 18 个月前只有一个顶级模型 (GPT-4)，如今有多个旗鼓相当的选择

频繁迭代也带来版本管理难题。Vertex AI 的特性之一是在同一个 Agent pipeline 中定义多个版本（可切换），并记录 A/B 实验效果。这样即使某个版本出现 regress，也能迅速回滚。

2.4 多模态融合

- 单模态模型正在被多模态模型取代（Gemini 原生支持文本、图像、音频、视频）
- 企业对多模态输入（文档中的图表、视频内容分析）需求日增

多模态融合意味着要同时处理数据同步、对齐和分辨率问题。Gemini 在语音、图像、文本之间建立统一 embedding space，使得一个输入流可以同时被多个 tokenizers 解析，避免跨模态断层。

2.5 成本与延迟持续下降

- API 调用成本已下降约 1.5 个数量级
- 延迟下降使更多实时应用成为可能
- 稀疏 MoE 模型（只激活部分参数）进一步降低推理成本
- 有趣发现：运行某些 API 模型比运行等效开源模型更便宜（因省去运维成本）

成本下降后，企业更愿意走「算法 + 异构硬件」路线：在高峰期启用低延迟 GPU，低峰期开通 cheaper TPU/CPU，甚至把部分请求 offload 到 regional datacenter。这种做法要求 orchestration 层能感知实际请求特征。

2.6 本章小结

企业 GenAI 的核心趋势：AI 民主化、数据门槛降低、模型快速迭代、多模态融合、成本持续下降。

3 GenAI 技术演进时间线

3.1 2018-2021: foundation model 时代

这一阶段最大的突破是 Transformer 预训练 + 微调。Burak 讲到当时的 GTM：先用 BERT、GPT-2 解决 NLU/NLG 任务，再把 embedding 开放到企业 Search 与 document understanding。这个阶段的典型做法是 building an "AI co-pilot" for narrow tasks，强调「数据准备、feature engineering、knowledge graph」才是真正的竞争力。

3.2 2022-2023: Agent 与 multi-turn

Agent 的概念在 2022 年开始普及：LLM 不再只输出文本，而是 orchestrating tools。Vertex AI Agent Builder、LangChain 等框架出现后，工程师可以在几天内拼出 plan→tool→feedback 流程。这个阶段的痛点是「hard-coded workflow + manual monitoring」，所以 Burak 强调要做 metrics-first design。

这个阶段常见的 repository 包含：prompt templates、tool manifest、evaluation harness。Burak 特别提到：要把 tool 的 sample input/output, estimated latency, failure modes 都写成 doc 供 planner 参考。否则 agent 容易因误判 tool call 导致 cascade failure。

3.3 2024 之后：实时、可控、合规

进入 2024 年，企业关注点转向实时性与合规。Gemini 的 multi-modal、Vertex 的 agent scheduler、Anthropic 的 RSP，都对「model governance」提出更高要求。新阶段的关键词是「可追溯」、可视化「policy-as-code」与严谨的「failure analysis」。

一个关键趋势是「legal hold + audit trail」逐渐成为 baseline。合规团队要求每个 decision path 都可以 reproduce，甚至要把 user prompt 与 tool call 串成 timeline 供审查。很多企业把这挂到 data lake，和 observability pipeline 联动。

3.4 本章小结

回顾时间线可以看到：从预训练到 agent，再到合规，企业 AI 的挑战一路从 capability 转向 control，这意味着团队必须不断迭代流程与组织结构。

4 企业 GenAI 典型实践

4.1 金融服务中的智能助理

数字银行常见的 GenAI 路径是：把 LLM 嵌入客户服务、财富管理和内部合规流程。典型模式如下：

- **输入**：用户询问投资建议 / 举报可疑交易
- **搜索**：实时抓取市场数据、监管规定
- **LLM**：生成解释性答案 + 操作建议
- **工具**：自动填报报告、生成合同草稿

金融 AI 的三层闭环

1. 前端：LLM 生成解释性答复
 2. 中间层：动态搜索/检索确保信息实时
 3. 后端：低代码自动化工具执行命令
 三层之间要保证可追溯性（为什么推荐这条策略？）与风险防范（拒绝敏感操作）。

4.2 制造与供应链的 GenAI Agent

制造公司用 GenAI Agent 连接 MES、ERP、供应链可视化工具：

- 工程师上传 CAD + 用例描述
- Agent 读取 BOM、测试记录、故障历史，生成排查计划
- 直接触发工单或通知采购规划

测试场景必须真实再现

很多 Agent 在测试环境表现良好，但一进真实车间就崩溃是因为数据分布差异（日志 vs. 实时 sensor）。必须在真实操作中收集 feedback 并回放给模型。

4.3 本章小结

不同领域的实践都遵循同样的闭环模式：AI 提供理解 + 生成，搜索/数据确保事实，工具/流程完成执行，且整个链路需要可追溯和可控。

5 搜索 + LLM：企业的杀手级应用

Grounding with Search

几乎每个企业用例都在结合 LLM 和搜索：

- LLM 的知识有时效性限制——需要搜索获取最新信息
- LLM 会产生幻觉——需要搜索做事实验证
- LLM 擅长推理但缺乏引用——搜索提供精确的来源引用

这使得 RAG（检索增强生成）成为企业 AI 应用的标配架构。

从工程角度看，RAG pipeline 需要处理三个关键问题：一是向量索引增长带来的 latency，二是检索异常（返回过时或错误文档），三是 LLM 与检索结果的融合策略。Burak 讲到一些团队会在检索结果前加一层「fact filter」——把检索结果先交给 smaller model 进行精排，再交给大型 LLM 生成。

5.1 本章小结

搜索 + LLM 是企业最普遍的 GenAI 应用模式，解决了时效性、幻觉和引用三大痛点。

6 构建 AI Agent 的关键模块

6.1 Vertex AI 平台架构

Google Vertex AI 的设计体现了企业 AI 平台的共性需求：

- 芯片层：GPU/TPU 选择灵活性（当前全球芯片严重短缺）
- 模型层：第一方 + 第三方 + 开源模型的统一接口
- 工具层：Model Builder 和 Agent Builder 提供定制和编排能力
- 应用层：搜索、对话、数据分析等垂直应用

6.2 Gemini 的差异化

- 原生多模态（从训练开始就混合多种模态）
- 超大上下文窗口（实验中已达 1000 万 token）
- Needle-in-a-haystack 测试：在超长上下文中精确检索信息

6.3 Planner 与 Memory 模块

Agent 内部通常包含 planner、executor、memory 三个协同组件。典型流程是：

- **Planner**：把用户意图拆解为步骤（比如搜索、工具调用、总结）
- **Memory**：记录每次对话、查询与工具输出，提供 context re-use
- **Executor**：负责调用特定工具、API 或自定义运行时

planner+memory 的同步挑战

当 planner 制定步骤后，memory 需要快速返回过去的执行结果。如果 memory 不够精细（只保留最近一轮），planner 会重复调用工具；如果 memory 太大，又会造成 latency。解决办法是引入 compressed memory + recall hints，即在 memory 中维护 query summary 供 planner 参考。

Planner/Memory 的分工

Memory 为 planner 提供上下文缓存，planner 决策要素（上一轮失败、工具成功率），executor 在安全 guardrail 之下运行。这个三层架构让 agent 能在长上下文、跨工具的任务中保持连贯。

6.4 Tool Orchestration 与 Workflow

Agent 要协调多个工具（搜索、数据库、API），需要明确工作流和 tool specification：

- 定义 tool intent、inputs、outputs，避免 model hallucinating tool usage
- 用 ‘tool descriptor’ 验证 LLM 的调用参数（类型、范围）
- 在 workflow 中插入 ‘pre-conditions’ 与 ‘post-conditions’

workflow 的 state machine 设计

Complex agent often models workflow as state machine：每个状态代表一个任务阶段（理解、planning、execution、confirmation），transition 触发条件来自 planner 的判断。这样可以在开发阶段用 unit test 模拟不同分支，提高 system reliability。

tool descriptor 的价值

当模型需要调用 ‘fetch_legislation()’ 或 ‘run_cad_simulation()’ 时，tool descriptor 提供 schema、authority、expected latency，避免 agent 盲目调用或超时。

6.5 本章小结

企业 AI 平台需要在芯片、模型、工具和应用各层提供选择灵活性。

7 AI Agent 的可观测性与安全控制

7.1 监控指标与报警策略

一个成熟的 Agent 平台首先要定义可观测指标，然后在「信号」出现时自动告警：

- **语义一致性**：Historical Reference vs. current response divergence
- **模型幻觉率**：随机抽查输出，看是否引入虚假事实
- **工具调用错误**：命令执行失败 / API 调用超时
- **上下文暴涨**：上下文窗口突然膨胀说明 memory leak 或 prompt injection
- **用户纠错次数**：用户频繁打断说明 agent 误解 intent

越早捕捉 signals 越容易纠偏

很多团队等到用户投诉才知道 agent 出问题了。早期的异常信号往往隐藏在 latency spike、工具调用超时或“内容重复”中。设置透明的「metric + 增强 sampling + human-in-loop review」可在 agent 出错前升级响应。

Burak 强调 instrumentation 的数据 pipeline 要保持低开销：对语义一致性或 hallucination 只采样 1-2% 请求，通过 squad 评审保持敏感性。过度采集会产生成本，也可能导致告警疲劳。

7.2 安全治理与 guardrail 设计

Agent 的行为不能只靠模型 prompt 设定，还要靠系统级 guardrail：

- **Policy Layer**：在调用前拦截敏感操作（删除文件、调度任务）
- **Tool Firewall**：将模型对工具的访问限制在白名单
- **Human-in-the-loop level**：根据任务风险动态决定是否需要人工审批
- **Audit Trail**：记录每一步决策、工具调用和用户确认

没有 audit trail 就无法追责

一旦 agent 造成错误或安全事件，缺少操作日志意味着无法恢复决策链。这会导致治理系统失效，甚至影响合规审计。Anthropic 和 Vertex AI 都强调：每一步 tool call 都要可追溯、可还原。

7.3 本章小结

通过量化指标、报警链路和责任链路，企业才能在 agent 出现异常时迅速反应，并在能力上升的同时保持安全边界。

8 AI Agent 的部署与运营

8.1 成本与延迟 trade-off

部署 Agent 不仅是模型选择，也是 resource orchestration：

- **推理架构**：chat + tool loop vs. batch prompt，不同策略对 latency 影响巨大
- **资源阶梯**：热 path 使用 low-latency GPU，冷 path 使用 cheaper CPU
- **Auto-scaling**：按 request rate 自动扩缩容，避免 overprovisioning
- **Multi-model routing**：低风险任务走 smaller model，高风险任务走 Sonnet/Opus

不能只看吞吐，要看 end-to-end experience

一个 Agent 的成功不是单纯推理速度，而是整体任务完成时间：LLM 回答 → API 调用 → 后处理 → 用户确认。任何一步延迟都能破坏用户体验。Vertex AI 的 Agent Builder 通过 pipeline 仪表板，把每个组件 latency 具体归集到 dashboard。

这里的经验是：Pod 规模和 prompt size 会构成两个相互挤压的维度。对于 token-heavy application，可以把 pipeline 分成「粗粒度 first pass」和「精细 second pass」，在保证 latency 的同时把 model call 费用最小化。

8.2 数据与用户反馈闭环

持续提升 Agent 需要循环利用 real user data：

- **Structured feedback**：插入「是否解决问题」「是否合规」的按钮
- **Failure tagging**：把失败案例打上标签（hallucination、tool failure、safety violation）
- **Retraining loop**：把高价值的 failure 推进 fine-tuning/LoRA
- **Prompt atlas**：每次 prompt 修改都版本化，便于回滚

Feedback loop 比「再训练」更早见效

很多企业先用 user feedback 细调 prompt，再决定是否重新训练。这种「先微调交互，再大规模 retrain」策略减少了迭代成本，也能更快交付可信 Agent。

在 Vertex AI 中，feedback loop 通常跑在独立的 ‘data labeling project’ 中：将失败案例发给 labeler 标注后，自动触发 retrain pipeline。这种自动化闭环保证 model loop 时刻保持 fresh。

8.3 本章小结

Agent 运营侧重延迟成本、资源配比和数据闭环；把这些环节标准化后，团队才能把实验室能力推向生产环境。

9 对学生和创业者的建议

- 研究方向：如何为特定用例定制大模型？如何用长上下文替代传统搜索？
- AI 领域最需要**创造力**——不仅是技术能力，还需要跨学科思维
- 建议在 AI 之外学习人文、艺术或其他科学，培养创造性思维

9.1 本章小结

学生与创业者最需要的是把技术好奇心转化为跨学科项目：融合产品、体验和社会价值，构建实际可以落地的 GenAI 服务，而非只写论文。

10 LLM Agent 的评估与实验室

10.1 定量评估指标

企业部署 Agent 之前需要构建一套定量值指标：

- **Task Success Rate**: 目标完成率（例如自动整理报告并获得审批）
- **Force Stop Rate**: 用户主动中止对话的比例
- **Tool Error Rate**: API/工具出现错误导致 workflow failure
- **Response Divergence**: 不同模型版本输出的一致性

这些指标通常按 SLA 分层：Critical Task 用 99.9% success，Exploratory Task 允许更低门槛。

10.2 Human-in-the-loop 实验室

Burak 提到企业会设立「Agent Lab」：小规模用户面向真实场景的 testing。流程如下：

1. 模拟真实任务，让 agent 在尽可能真实的系统中执行
2. 安排人工 reviewer 定期 audit output，打标签
3. 把 label 结果做成 dashboard，触发 retrain / prompt tune

Agent Lab 的组织价值

Agent Lab 是 bridge between research and product：研究团队可以看到真实 failure，产品团队可以体验新版本，治理团队可以验证安全 guardrails 是否有效。

10.3 本章小结

评估体系由定量指标与人类实验室组成，保证 agent 既能高效，也能在失败时迅速定位 root cause。

11 治理与法规对接

11.1 政策与合规需求

GenAI/Agent 在企业中经常触及金融、医疗、政府等敏感领域。需要同时满足以下监管要求：

- **数据主权**：数据必须在指定区域处理，不能跨境泄露
- **透明性**：用户要能看到为什么 agent 做出某个推荐
- **安全审计**：所有 tool 调用需记录并可供合规团队审查
- **偏见检测**：agent 不能在种族/性别上产生歧视性输出

11.2 技术与治理结合

合规不是单靠文档，而是要把 guardrail 编成 code：

- **policy-as-code**：把规则写成可执行 policy（如 Open Policy Agent）
- **simulation sandbox**：每次改 prompt 先跑 sandbox 测试，确保不会违反 policy
- **kill-switch**：当检测到攻击/滥用时，agent 可自动进入安全 mode

监管不是 checkbox，而是持续对话

监管机构会根据实际应用不断提出新要求。企业需要把 regulator view 嵌入 daily standup——当出现 new capability 时，先评估是否触发新的 policy，再上线。

11.3 本章小结

治理需要跨组织协作：法律、合规、AI 安全、工程共同构建 policy-as-code、杀手开关和可解释的审计 trail。

12 组织与投资的启示

12.1 人才结构与文化

Burak 特别强调人才密度与迭代文化的重要性：把跨功能 squad 组织成 product + safety + research 的三位一体。每个 squad 内部都需要 prompt engineer、search engineer、ops engineer 与 policy analyst 四类角色，确保 agent pipeline 不仅能跑，还能管。

Anthropic 在 2023 年的经验是：人才密度 > 数量。他们把组织值定义成「谁能马上跑完实验并解释 failure」。这种文化确保 code review 同时覆盖 quality 与 compliance。

12.2 资本分配与项目组合

GenAI 投资不是全部押在一个 agent，而是构建多阶段 portfolio：

- **Stage 1**：GPT-style capability research 继续探索 scaling, architecture
- **Stage 2**：Platform / Agent ops 提供 tooling + governance

- Stage 3: Productization 把 agent 变成 specific vertical solutions (如 finance assistant)

分阶段 portfolio 的好处

这样的组合允许团队在 early-stage 上做创新探索，mid-stage 保证 platform reliability，late-stage 担当 commercialization。每个阶段的 metrics / budget 都需要清晰分离，避免能力部门和 product 部门在同一个 sprint 中彼此干扰。

12.3 本章小结

组织层面需要建立「人才密度」「跨功能 squad」和「分阶段投资」三大机制，才能在技术快速演进之际保持团队可控。

13 关键幻灯片回顾

13.1 封面与主题定位



图 1: 课程封面：从企业视角审视 GenAI 趋势

封面指出：本讲不是算法研究，而是从企业 GTM、平台、工具、治理四个层面切入。这也决定了整套幻灯片在每个主题都含案例、数据、工具链、政策四个维度。

13.2 趋势图谱与 timeline

幻灯片中多处出现趋势图谱：左侧列出技术 capability (如 multi-modal, agent)，右侧列出企业需求 (如 compliance, observability)。Burak 反复提醒要把这张图挂在墙上，当团队做 roadmap 时参照。

趋势图谱的使用方法

1. 先定位你的 capability: 是否具备 search + tool loop?
2. 再看需求右侧: 哪些 regulator、which SLA 还没满足?
3. 最后画出 gap: gap 本身就是 prioritization 依据

13.3 实时性与可观察性展示

另一张重点幻灯片展示了 Vertex Agent 的 observability dashboard, 左侧是 latency/time series, 右侧是 tool-call matrix。Burak 指出, dashboard 不是 nice-to-have, 而是 product launch 条件之一。

13.4 本章小结

幻灯片不仅提供宏观框架, 更可直接复用到团队 workshop (趋势图谱 + dashboard 模板)。

14 Agent Workflow 深度剖析

14.1 发现与理解

每个 agent 都以 intent detection start: 把 natural language 输入映射到 structured goal。团队会为常见 intent 预定义 slot template。Burak 介绍的实践是先用 small model 做 classification, 再用 large model 做 planning。

14.2 规划与工具选择

planner 既可以是 chain-of-thought prompt, 也可以是 learned policy。Burak 展示了一个 hybrid approach: 先用 policy 生成 plan tree, 再用 LLM 依次展开。关键在于 tool selection module: 它要考虑 tool current status (是否健康)、estimated latency 以及 user context。

14.3 执行与复核

Executor 其实就是 tool orchestration layer。除了调用 API, 还要 handle retries、fallbacks 和 confirm-with-user。Burak 强调: 对于金融类操作, 一定要加入 explicit confirmation step, 并把决策记录在 audit log 中。

14.4 收尾与迭代

Workflow 的最后一个阶段是总结和 logging: 生成 summary 给用户、更新 memory、把 all interactions 记录到 observability pipeline, 并根据 failure tag 触发 training pipeline。

14.5 本章小结

把 agent workflow 拆解成发现、规划、执行、收尾四步, 有助于在每个阶段落地指标、监控与审查。

15 治理案例：监管 playbook

15.1 金融监管 playbook

Burak 重点强调金融场景的监管要求：每一个 recommendation 都要附带 fact sheet，明确 cite 哪个 regulation。Agent pipeline 会自动生成 ‘regulatory footprint’ 供 compliance review。

15.2 医疗与政府的额外 guardrail

医疗机构要求系统提供 breach detection：agent 生成治疗建议前会先 check patient consent、enforce whitelist tool。这类 guardrail 通常 by policy-as-code 与 real-time monitoring 结合——形成「statement + enforcement」闭环。

15.3 本章小结

治理 playbook 由不同 vertical 的 policy 组成，所有 policy 都通过 code 化的 gatekeeper 落地，确保 agent 的能力不超出规则范围。

16 附录：Prompt 与 Tool 模板

16.1 Prompt Template 架构

Burak 推荐的 prompt template 包含三个部分：

1. Context：包含任务背景、policy 约束、用户意图
2. Instruction：明确当前步骤的输出要求（比如「列出 top-3 解决方案」）
3. Tool Usage：列出可选工具的调用方法，附带 success indicator

Prompt Template 的检查清单

每次更新 prompt 都要验证：1. Context 是否更新（policy / data）2. Instruction 是否仍然清晰 3. Tools & examples 是否可复现

16.2 Tool Manifest 示例

典型 tool manifest 包含以下字段：

- tool name
- description
- input schema（参数类型、可选值）
- latency range
- safety constraints（哪些 user group 不能用）

即便 tool 只是内部 API，也建议写成 manifest，物流团队可以当作操作手册。

16.3 Failure Tag 标签体系

Burak 建议统一 failure tag taxonomy:

- hallucination
- tool failure
- context expiration
- safety violation
- latency breach

每个 tag 关联处理流程: 例如 ‘tool failure’ 触发 fallback or escalate, ‘hallucination’ 触发 LLM retrace path。

16.4 本章小结

Prompt 与 tool 模板是 agent reproducibility 的基石, 把失败模式结构化之后, 产品和工程可以用更小的迭代去修复。

17 Q&A 与辩题

17.1 用户问答速览

访谈后半段 Burak 回答了听众关心的问题:

- **Q: 如何让 agent 在多语言语境中表现一致?**
A: 做好 localization pipeline, 把 prompt 和 tool manifest 都国际化, 使用 multilingual embedding 栈。
- **Q: 如何审查 agent 的 hallucination?**
A: 结合 search+LLM, 生成 answer 时同时输出 source cite, 并把 cite 和 user prompt 送入 evaluation harness。

17.2 辩题与反思

Burak 提出两个值得 debate 的命题:

1. Agent 的安全 guardrail 应该先行还是同步迭代?
2. Multi-modal 能力是否应该优先部署在低风险 task?

他的立场是: guardrail 需要同步迭代 (不能等能力再提升才加), multi-modal 应该先在 high-bandwidth observation 不敏感场景验证, 再逐渐扩展到高敏感度场景。

17.3 本章小结

Q&A 部分透露了听众最关心的 pain point, 也凸显出企业要在能力与 governance 之间持续 debate。

18 运营检查表

18.1 Deployment Readiness Checklist

在 Burak 的实践里，每次 agent 进入 production 前都会过以下 checklist：

- Metrics pipeline 持续采样 Task Success、Tool Error、Latency
- Observability dashboard 已经接入 DataDog / Vertex monitoring
- Policy guardrails 的 intercept tests 通过
- Feedback loop pipeline 可以自动把 failure 标注并触发 retrain

18.2 Post-launch Monitoring

上线后以 sprint 为周期，每周审查：

- alert channel 是否有重复的 failures? 需要 adjust thresholds
- new user feedback 是否泄露 policy violation
- tool reliability (#timeout, #fallback) 是否稳定

只有持续监控才能保证 agent 没有 drift。

18.3 本章小结

运营检查表把技术、治理、监控融合成 repeatable process，是把实验室能力推向生产的关键。

19 未来展望与行动建议

19.1 未来 12 个月的路线

Burak 预测未来一年将聚焦两个方向：1) 将 agent 统一到 single platform (不再用碎片化 stack)；2) 提升 governance capability，尤其是 multi-modal output 的 auditability。

19.2 战略性建议

- 把 agent capability roadmap 写成 multi-layer diagram (capability + policy + Ops)
- 在 early stage invest in tool manifest & prompt atlas，而不是直接 GPT-4 最终 prompt
- 采用 ‘policy-as-code’ tooling 让 compliance team 参与 sprint planning

19.3 本章小结

未来一年会把 attention 从 capability 升级到 governance 与 Operations，提前准备的团队会赢在落地速度。

20 术语速查

20.1 核心术语表

术语	解释
Agent Builder	Vertex AI 的可视化工具，用于定义 planner、tools、feedback loop
Policy-as-code	用可执行代码定义政策规则，结合 OPA 等执行 gatekeeping
Tool manifest	对每个工具的输入、输出、latency、failure mode 进行描述
Feedback loop	把 user feedback、failure tags 送到 labeling & retrain pipeline
Observability pipeline	聚合 latency、tool usage、alerts 到 dashboard

术语表的作用

对内：帮助 onboarding 的工程师快速理解平台；对外：给 compliance 团队提供统一定义。建议把术语表挂在 README 并定期维护。

20.2 应用与维护建议

- 把 glossary 版本控制，与 README 同步，确保只要 policy 变化就更新术语
- 新人 onboarding 时让他们复述 glossary，确保理解与记忆
- 把 glossary 转成 ‘cheatsheet’ 供 compliance meeting 与 board review 使用

21 附加素材与灵感

21.1 组合的思路

将本讲内容组合成 workshop 时可以分成三个环节：1) Trends Review 以 timeline + trends table 起手；2) Platform Deep Dive 讲 Vertex + Observability；3) Governance Sprint 用 prompt template / policy-as-code 作 hands-on exercise。每个环节都可以配一页 slide，让团队在 3 小时内复盘全部内容。

21.2 引用与灵感

Burak 多次提到「观察力比新模型更重要」：保持 instrumentation、dashboarding、alerting，能让你在模型升级时及时发现 regress，而不是等用户投诉。这个观点也反映在 Table 1 的 risk column 中：observability 直接降低了 compliance risk。

21.3 本章小结

附加素材帮助团队把 talk 的内容快速变成 workshop 与 playbook，让 narrative 不只是听完就忘，而是真的落地。

22 总结与延伸

22.1 课程全局总结表

主题	课程给出的结论	关键行动	风险 / 关注点
企业 GenAI 趋势	AI 民主化与平台化正在扩大用户	构建多模态平台、准备弹性 tooling	依赖单一模型或 API 易造成 lock-in
Agent 架构	search + LLM、Vertex AI 工具链是实作基座	细化 memory、retriever、planner 等模块	崩溃成本高、工具调用错误频发
可观测与治理	指标、告警、审计 trail 是部署前提	建立 policy layer 和 human-in-loop workflow	没有 audit trail 难以追责
部署运营	latency+cost tradeoff 需要精细调度	multi-model routing、auto scaling 与 feedback loop	过度扩容会浪费预算，欠扩容会影响体验
人才与创新	需要创造力与跨学科视角	投资 Prompt 迭代、跨领域项目、社会价值	只做技术研究会错过市场与社会影响

表 1: Lecture 09 的核心结论与行动对照

22.2 进一步行动与反思

- 对企业产品团队：把 search 与 LLM 的组合做成一个闭环 pipeline，再嵌入质量监控与 feedback loop
- 对部署团队：把 tool 使用、latency、cost 做成常态仪表板，并让模型更新都在 sandbox 中先运行
- 对安全治理：把 Policy Layer 与 audit trail 集成，确保 agent 出错时能快速回溯
- 对研究者与学生：把创造力与跨学科视角融入 AI 项目，关注社会价值而非纯技术

22.3 拓展阅读

- Google, “Gemini: A Family of Highly Capable Multimodal Models,” 2023.
- Google Cloud, “Vertex AI Agents Documentation,” 2024.
- Anthropic, “Responsible Scaling Policy,” 2023.
- Kamradt, “Needle in a Haystack - Pressure Testing LLMs,” 2023.